
1 First: the REAL error in your code

You wrote:

```
const newArr = arr.filtering((num)=>num>25);
```

But **arr** is never defined ❌

You commented it out:

```
// this = [10,20,30,5,90,87];
```

✅ Correct version

```
const arr = [10, 20, 30, 5, 90, 87];
```

2 Why did we NOT write **this = []** ?

You asked:

"why did we do this"

(meaning: why are we using **this**, and why is **this = []** wrong?)

❌ **this = []** is illegal JavaScript

this is NOT a variable

You cannot assign to it.

```
this = []; ❌ SyntaxError
```

Because:

- **this** is a keyword
- It is automatically provided by JavaScript

- You can **read** it, not **overwrite** it
-

3 What does **this** mean here?

Your method:

```
Array.prototype.filtering = function(compare) {  
  const ans = [];  
  for (let num of this) {  
    if (compare(num)) {  
      ans.push(num);  
    }  
  }  
  return ans;  
};
```

 **Key idea:**

 Inside a prototype method, **this** refers to the array that calls it

4 Step-by-step execution (VERY IMPORTANT)

Code:

```
const arr = [10, 20, 30, 5, 90, 87];  
const newArr = arr.filtering(num => num > 25);
```

What JS does internally:

```
arr.filtering(...)  
↑  
this === arr
```

So inside **filtering**:

```
this === [10, 20, 30, 5, 90, 87]
```

Loop becomes:

```
for (let num of [10, 20, 30, 5, 90, 87]) {
```

That's why we use **this** instead of passing the array manually

5 Why we create **ans = []** instead of modifying **this**

Because **filter** should not change the original array

✗ BAD (mutates original array)

```
this.push(num);
```

✓ GOOD (returns new array)

```
const ans = [];  
ans.push(num);
```

Same behavior as native **.filter()**.

6 Why NOT use arrow function for prototype method?

✗ Wrong:

```
Array.prototype.filtering = (compare) => {  
  console.log(this); // ✗ window / undefined  
};
```

s

Arrow functions:

- ❌ Do NOT have their own `this`
- ❌ `this` comes from outer scope

✅ **Correct:**

```
Array.prototype.filtering = function(compare) {  
  console.log(this); // ✅ array  
};
```

7 FINAL FIXED CODE (WORKING)

```
const arr = [10, 20, 30, 5, 90, 87];  
  
Array.prototype.filtering = function(compare) {  
  const ans = [];  
  for (let num of this) {  
    if (compare(num)) {  
      ans.push(num);  
    }  
  }  
  return ans;  
};  
  
const newArr = arr.filtering(num => num > 25);  
console.log(newArr); // [30, 90, 87]
```

8 One-line mental model 🧠

`this` = the object on the LEFT of the dot

```
arr.filtering(...)  
↑  
this === arr
```

1 What does “this” even mean?

In JavaScript, **this** is decided at **CALL TIME**, not where the function is written.

👉 It answers the question:

“Who called me?”

2 Normal functions — **this** is DYNAMIC

Example:

```
function show() {  
  console.log(this);  
}
```

```
show();
```

Output:

- In browser → **window**
- In strict mode → **undefined**

Now:

```
const obj = {  
  name: "JS",  
  show: show  
};
```

```
obj.show();
```

Output:

obj

🔑 Same function, different **this**
Because **who called it** changed.

3 Arrow functions — **this** is FIXED

Arrow functions **DO NOT** decide **this** at call time.

Instead:

Arrow functions **CAPTURE** **this** from where they are **WRITTEN**

This is called **lexical this**.

4 Simple visual rule

Function type	How this is decided
Normal function	Who calls it
Arrow function	Where it is written

5 PROOF with code (MOST IMPORTANT PART)

◆ Normal function inside object

```
const obj = {
```

```
    name: "JS",
    sayHi: function () {
      console.log(this.name);
    }
  };
```

```
obj.sayHi(); // JS 
```

Because:

```
obj.sayHi()
↑
this === obj
```

◆ Arrow function inside object

```
const obj = {
  name: "JS",
  sayHi: () => {
    console.log(this.name);
  }
};
```

```
obj.sayHi(); // undefined 
```

WHY??

Because arrow function looks OUTSIDE:

```
const obj = {
  sayHi: () => {
    console.log(this); //  NOT obj
  }
};
```

It checks:

“What was `this` outside this function?”

Outside is:

- Global scope
- `this === window` (browser)

So:

```
window.name // usually empty
```

6 YOUR prototype case (VERY IMPORTANT)

You wrote:

```
Array.prototype.filtering = (compare) => {  
  console.log(this);  
};
```

What is `this` here?

Arrow function says:

“I don’t care who calls me. I’ll take `this` from outside.”

Outside is:

```
global / window
```

So:

```
this !== arr ❌
```

That’s why **arrow functions BREAK prototype methods.**

✓ Correct prototype method

```
Array.prototype.filtering = function (compare) {  
  console.log(this); // ✓ the array  
};
```

Now:

```
arr.filtering()  
↑  
this === arr
```

7 One more crystal-clear example

✗ Arrow breaks **this**

```
const obj = {  
  value: 10,  
  inc: () => {  
    this.value++;  
  }  
};
```

```
obj.inc();  
console.log(obj.value); // 10 ✗
```

✓ Normal function works

```
const obj = {  
  value: 10,  
  inc() {  
    this.value++;  
  }  
};
```

```
obj.inc();  
console.log(obj.value); // 11 ✓
```

8 WHY arrow functions were created 🧠

They were made to **FIX THIS PROBLEM**:

```
function Timer() {  
  this.seconds = 0;  
  
  setInterval(function () {  
    this.seconds++; // ✗ this = window  
  }, 1000);  
}
```

Old hack:

```
const self = this;
```

Arrow solution:

```
function Timer() {  
  this.seconds = 0;  
  
  setInterval(() => {  
    this.seconds++; // ✓ inherits this from Timer  
  }, 1000);  
}
```

9 FINAL RULES TO MEMORIZE 🔥

✗ **NEVER** use arrow functions for:

- Prototype methods
- Object methods that use `this`
- Event handlers needing element `this`

✓ **USE arrow functions for:**

- Callbacks
- `setTimeout` / `setInterval`
- `map` / `filter` / `reduce` callbacks

ONE-LINE SUMMARY

Arrow functions don't HAVE `this` — they BORROW it from where they are written

✓ **YES — your understanding is correct**

Arrow function ⇒ `this` refers to where the function is *defined*

Normal function ⇒ `this` refers to who *calls* the function

Now let's lock this in with **clear side-by-side examples** 📌

1 SIMPLEST POSSIBLE EXAMPLE

◆ Normal function

```
function show() {
```

```
    console.log(this);  
  }  
  
  show();
```

📌 Output:

- browser → `window`
- strict mode → `undefined`

Because:

“Who called me?” → global

◆ Arrow function

```
const show = () => {  
  console.log(this);  
};  
  
show();
```

📌 Output:

- Same `this` as **outside** this function

👉 Arrow did **not decide** `this`, it copied it

2 OBJECT METHOD (MOST COMMON CONFUSION)

✓ Normal function (correct)

```
const user = {  
  name: "Hari",  
  greet: function () {  
    console.log(this.name);  
  }  
};
```

```
user.greet(); // Hari
```

Because:

```
user.greet()  
↑  
this === user
```

✗ Arrow function (wrong here)

```
const user = {  
  name: "Hari",  
  greet: () => {  
    console.log(this.name);  
  }  
};
```

```
user.greet(); // undefined
```

Why?

```
this → taken from where function is defined  
this → global (window)
```

3 PROTOTYPE METHOD (YOUR CASE)

❌ Arrow breaks prototype

```
Array.prototype.myFilter = (cb) => {  
  console.log(this);  
};  
  
[1,2,3].myFilter(); // ❌ window / undefined
```

✅ Normal function works

```
Array.prototype.myFilter = function (cb) {  
  console.log(this);  
};  
  
[1,2,3].myFilter(); // ✅ [1,2,3]
```

4 CALLBACK INSIDE METHOD (WHERE ARROW SHINES)

❌ Normal function loses **this**

```
const counter = {  
  count: 0,  
  start() {  
    setInterval(function () {  
      this.count++;  
      console.log(this.count);  
    }, 1000);  
  }  
};  
  
counter.start(); // NaN / error
```

Because:

```
setInterval(...)  
↑  
this === window
```

✓ Arrow FIXES it

```
const counter = {  
  count: 0,  
  start() {  
    setInterval(() => {  
      this.count++;  
      console.log(this.count);  
    }, 1000);  
  }  
};
```

```
counter.start(); // 1, 2, 3...
```

Arrow says:

“I’ll use `this` from `start()`”

5 EVENT HANDLER (IMPORTANT)

✗ Arrow loses element reference

```
button.addEventListener("click", () => {  
  console.log(this); // ✗ window  
});
```

✓ Normal function gives element

```
button.addEventListener("click", function () {  
  console.log(this); // ✅ button  
});
```

6 .call, .apply, .bind (arrow ignores them!)

Normal function

```
function show() {  
  console.log(this.name);  
}  
  
show.call({ name: "JS" }); // JS
```

Arrow function

```
const show = () => {  
  console.log(this.name);  
};  
  
show.call({ name: "JS" }); // ❌ still undefined
```

Arrow:

"I don't care about call/apply/bind"

7 VISUAL SUMMARY TABLE 🧠

Case

Normal Function Arrow Function

Has its own <code>this</code>	✓ Yes	✗ No
<code>this</code> decided at call time	✓	✗
<code>this</code> from definition scope	✗	✓
Good for object/prototype methods	✓	✗
Good for callbacks	✗ sometimes	✓

🔑 FINAL ONE-LINE RULE (MEMORIZE THIS)

Normal function: `this = who called me`

Arrow function: `this = where I was written`

✗ What is wrong?

```
console.log(count)
```

🚫 **`count` is NOT defined in this scope**

Inside `increment`, you correctly updated:

```
this.count++
```

But then you logged:

```
count ✗
```

There is **no variable named `count`** in the function or global scope.

✓ Correct code

```
const counter = {  
  count: 0,  
  increment: function () {  
    this.count++;  
    console.log(this.count);  
  }  
};  
  
counter.increment(); // 1
```

🧠 Why this happens (VERY IMPORTANT)

Inside object methods:

- `count` ✗ → looks for a local or global variable
 - `this.count` ✓ → accesses property of the object
-

🔍 What JS tries to do internally

```
console.log(count);
```

JS looks for:

1. `count` inside `increment` ✗
2. `count` in outer scope ✗
3. `count` on `window` / `globalThis` ✗

➡ **ReferenceError: count is not defined**

Proof

```
let count = 100;
```

```
counter.increment(); // logs 100 ❌ (WRONG LOGIC)
```

Because:

```
count !== this.count
```

Mental rule (memorize this)

Object properties must ALWAYS be accessed via **this** (or object name)

Related trap with arrow functions (just awareness)

```
increment: () => {  
  this.count++;      // ❌ this is not counter  
}
```

Arrow would break **this**.

Best practice (modern syntax)

```
const counter = {  
  count: 0,  
  increment() {  
    this.count++;  
    console.log(this.count);  
  }  
};
```

1 First: fix the syntax (small typo)

You wrote:

```
start : () {
```

That's invalid JS.

Arrow functions must be:

```
start: () => {
```

So the **actual code** is:

```
const counter = {  
  count: 0,  
  start: () => {  
    setInterval(() => {  
      this.count++;  
      console.log(this.count);  
    }, 1000);  
  }  
};
```

```
counter.start(); // NaN
```

Now let's explain **WHY this gives NaN**.

2 Key rule (the root cause)

! Arrow functions do NOT have their own **this**

! They take **this** from **where they are defined**, not who calls them

You already said this — **correct so far** ✓

3 Where is **start** DEFINED?

This is the crucial part.

```
start: () => {
```

This arrow function is defined:

- ❌ NOT inside `counter`
- ❌ NOT bound to `counter`
- ✅ In the **outer (global / module) scope**

Objects **do NOT create scope** in JavaScript.

So this arrow function asks:

“What is `this` OUTSIDE me?”

Answer:

- Browser (non-strict): `this === window`
- Module / strict mode: `this === undefined`

4 Call chain (line by line)

Step 1: Call

```
counter.start();
```

⊘ This does **NOT** matter for arrow functions
They ignore the caller.

Step 2: Inside `start`

```
this === window    // or undefined
```

So now this line runs:

```
this.count++;
```

Which becomes:

```
window.count++; // or undefined.count++
```

5 Why exactly NaN?

Let's assume browser (non-strict):

```
window.count === undefined
```

Now JS does:

```
undefined + 1 === NaN
```

So:

```
window.count = NaN  
console.log(NaN)
```

💥 That's the NaN

6 What about the inner arrow in setInterval?

```
setInterval(() => {  
  this.count++;  
}, 1000);
```

That arrow inherits **this** from **start**.

But **start** itself already had the **wrong this**.

So this is what happens:

```
inner arrow
  ↑
takes this from start
  ↑
start took this from global
```

👉 Upward chaining **is happening**, but it ends at **global**, not **counter**.

Your intuition was right — just the **anchor point was wrong**.

7 Visual mental model 🧠

❌ Your broken version

```
global scope (this = window)
  ↑
start: () => { }   ← arrow steals this from here
  ↑
setInterval arrow ← steals same this
```

✅ Correct version

```
const counter = {
  count: 0,
  start() {           // NORMAL function
    setInterval(() => { // ARROW callback
      this.count++;
      console.log(this.count);
    }, 1000);
  }
};

counter.start(); // 1, 2, 3...
```

Visual:

```
counter.start() → this = counter
    ↑
inner arrow inherits this = counter
```

8 Final rules to LOCK IN 🔒

✗ Never use arrow for:

- Object methods
- Prototype methods
- Anything that needs `this`

✓ Use arrow for:

- Callbacks
- Timers
- `map / filter / reduce`